

StockPulse - Current Technical Architecture

1. Purpose

This document describes the current StockPulse implementation, not an old proposal or a future architecture. It should be kept aligned with the Flutter codebase and the live Supabase schema/functions.

Current stack: Flutter + GetX + Supabase (Auth, Postgres, Storage) + GetStorage.

StockPulse is a shop/POS management application covering the implemented operational flow around authentication, shop setup, role-based access, products, purchases, sales, expenses, reports, stock and printing.

2. Current Application Flow

```
APP START
  v
Supabase initialization
  v
Deep-link / password-recovery handling
  v
Session + recovery-state check
  |- recovery link -> verify recovery -> Create New Password
  |- no session    -> Login
  \- valid session -> load current shop membership / role
                        v
                    RoleService
                        v
                    Dashboard
                        v
    permitted tabs/actions based on owner/staff role
```

The app must not rely only on whether `Supabase.auth.currentSession` is non-null. Recovery state and current user's shop membership/role must also be resolved before normal dashboard routing.

3. Authentication & Password Recovery

Normal authentication

- Supabase Auth is the authentication source.
- Login/logout are handled through the auth layer/controller/repository used by the app.
- User-scoped GetX state must be cleared when logging out or switching authentication context so data from a previous account is not retained in memory.

Password recovery

StockPulse supports password reset through the verified HTTPS App Link:

`https://rishtajourney.com/reset-password?token_hash=...&type=recovery`

Current recovery behavior:

1. App receives the deep link through `app_links`.
2. Recovery token is verified using the token hash flow.
3. Recovery works whether the app was logged out or another account was already logged in.
4. Normal cached/user-scoped state must not leak into the recovery session.
5. A persistent `recovery_in_progress` state protects cold-start routing.
6. If recovery was started but the app was killed before password completion, the next launch must not route the recovery session into the normal dashboard.
7. After a successful password update, recovery state is cleared and the user returns to Login.
8. Expired/invalid links (`otp_expired`) are handled as recovery errors instead of being allowed to fall through to normal app routing.

Android App Links are configured for `rishtajourney.com / www.rishtajourney.com`, with `/.well-known/assetlinks.json` deployed and verified.

4. Shop, Membership & Roles

The old assumption that StockPulse has one global `shop_profile` row and needs no shop identifier is obsolete.

shops

The current database uses a `shops` table. Known current fields include:

- `id` - bigint primary identifier
- `created_at`
- `address`
- `shopename`
- `ownerame`
- `phone`
- `selectedsymbol`
- `selectedcurrency`
- `shopimg`
- `createdat`
- `updatedat`

Code and policies must use the actual current column names/types. In particular, `shops.id` is bigint; foreign keys referencing a shop must use a compatible type.

Membership / role state

Role resolution is centralized through `RoleService` rather than being scattered across screens.

Current role state includes:

role
hasMembership
isActive
shopId
shopName
isLoading

Core role helpers include:

isOwner
isStaff
canManageProducts
canCreateSale
canManagePurchases
canManageExpenses
canManageStaff

Current baseline behavior:

- Owner has management access.
- Staff has restricted access.
- Staff can create sales.
- Product, purchase, expense and staff-management access is owner-restricted unless the permission model is deliberately expanded.
- Navigation visibility is derived from centralized role state.
- Security must also be enforced in Supabase/RLS/RPCs; hiding a tab is not authorization.

The membership/profile data must preserve the actual authenticated user ID and display name so sales created by staff show the staff creator's name, not the currently logged-in owner's name when the owner later views the sale.

5. Dashboard & Navigation

DashboardScreen builds its available tabs from role-aware configuration rather than assuming every user sees every module.

Primary areas currently used in the app include:

- Home
- Products
- Sales
- Purchases
- More / additional modules

Dashboard/Home also provides quick actions such as:

- New Sale
- Add Purchase
- Add Products
- Add Expenses

Actions and tabs must respect RoleService permissions.

6. Products & Categories

Products are a core stock source and are used by both purchase and sale transactions.

Known product fields currently used include:

- id
- article
- category_id
- color
- size
- barcode
- unit
- purchase_price
- sale_price
- current_stock
- min_stock_threshold
- product image field used by the current implementation

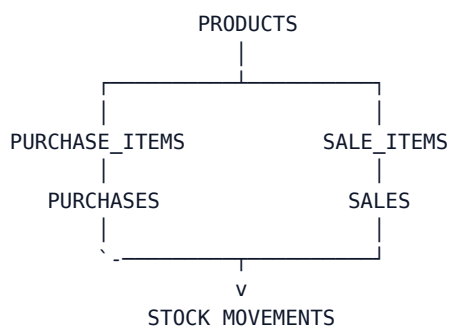
The UI supports product creation/editing, category selection, barcode-related flow, product images, stock values, pricing and low-stock behavior.

Product quantity is numeric so fractional quantities can be supported where the selected unit requires them; validation should be based on the product's unit/business rule rather than globally forcing integer quantities.

Image picking code must preserve the distinction between XFile and File; do not cast an XFile directly to File.

7. Stock Transaction Model

Current stock flow is centered on:



Stock-changing operations should remain server-controlled/atomic. The client must not independently create transaction rows and then separately adjust stock in a way that can leave partial state.

8. Sales

Sales are implemented through the Sale model/repository/controller flow.

Sale creation

SaleRepository calls the Supabase RPC used by the current application:

```
create_sale
```

The current RPC payload includes the implemented sale inputs such as:

```
p_items  
p_discount  
p_payment_method  
p_notes
```

SaleItemModel currently carries sale-line data including:

- productId
- article
- color
- size
- unit
- quantity
- salePrice

Sale creation must remain atomic with stock deduction and any server-side transaction records handled by the database function.

Sale UI/controller behavior

The Add Sale flow is a multi-step flow with top/bottom navigation and a save bar showing the running total.

Cart state includes quantities, sale prices, discount, payment method, notes and received amount where applicable. `clearCart()` resets transaction input state, and the Clear All action should be disabled when there are no selected quantities/items.

Sale creator

A sale must retain the actual creator user ID. When displaying sale details from another account (for example, owner viewing a staff-created sale), creator display name must be fetched from the profile/membership data for that stored user ID. It must not default to the viewer's current auth metadata.

Sale detail / printing

Sale detail supports invoice/printing actions. `SalePdfService.previewSale(...)` exists for PDF output, while the intended shop flow can also support direct thermal printing without forcing a PDF preview.

9. Purchases

Purchases use the Purchase model/repository/controller flow and the server-side purchase transaction/RPC already implemented in the project.

A purchase transaction must keep these changes consistent:

- purchase header
- purchase items
- affected product stock
- stock movement/history generated by the current database implementation

The Add Purchase screen uses its own bottom save bar (Save Purchase). Purchase history/report data is consumed separately from sale data.

Do not replace the current purchase implementation with an invented schema or client-side stock update flow.

10. Expenses

Expenses are now part of the current application architecture.

The current expense design uses fields including:

- id
- shop_id
- category
- description
- amount
- created_by
- created_at
- updated_at
- expense_date

Important schema rule: `expense.shop_id` must match the actual type of `shops.id` (bigint in the current shop schema). Do not define it as UUID.

The expense module consists of:

```
ExpenseModel  
ExpenseRepository  
ExpenseController  
Expense UI
```

The UI includes category metadata, amount/description/date presentation and dd/MM/yyyy formatting. Expense access is owner-controlled in the current role baseline.

RLS policies must be written against the actual current shop/membership relationship; do not reference nonexistent columns such as `shops.user_id`.

11. Reports

Reports are a current feature, not a future placeholder.

Implemented/current report work includes:

- Main Reports screen
- Sales Report
- Purchase Report
- Profit / P&L UI
- Stock reporting
- Date/filter-based reporting
- Loading shimmer states
- Dynamic chart axis labels/values based on report data

Relevant controllers include the report-specific controllers such as `SaleReportController` and `PurchaseReportController`. A detail/report screen must request the correct controller type; `SaleController` and `SaleReportController` are not interchangeable.

Report calculations should use persisted transaction data and server/query results rather than UI-only calculations that can drift from database truth.

12. Current Flutter Architecture

The existing StockPulse project structure is the baseline. Do not replace it with the old proposed feature-first features/.../services/... tree unless the project is intentionally refactored later.

Current high-level organization includes:

```
lib/  
|-- common/  
|   |-- route/  
|   `-- widgets/  
|-- controllers/  
|-- models/  
|-- repositories/  
|-- services/  
|-- utils/  
`-- views/
```

Important shared/current pieces include:

- `CustomAppBar`
- `CustomButton`
- `StandardScreen` / shared screen layout
- `AppConstants`
- `AppThemeHelper`
- centralized routes
- `GetX` controllers and reactive state
- repositories for Supabase data operations
- `RoleService`
- local-storage/recovery helpers
- PDF/printing services

Current architecture rule:

View -> Controller -> Repository -> Supabase / RPC

Some cross-cutting concerns live in services/ (for example role, deep-link/recovery, PDF/printing/local storage). There is no requirement to add a second generic data-service layer underneath every repository when the existing repository already owns that Supabase operation.

13. Important Controllers / Repositories / Services

The architecture should document actual classes as they exist rather than hypothetical interfaces.

Core controllers currently relevant

- DashboardController
- HomeController
- ProductController
- SaleController
- PurchaseController
- ExpenseController
- SaleReportController
- PurchaseReportController
- ShopCreateController
- ForgotPasswordController
- login/auth controller used by the current auth flow

Core repositories currently relevant

- ProductRepository
- SaleRepository
- PurchaseRepository
- ExpenseRepository
- ShopCreateRepository
- auth-related repository used by the current implementation

Cross-cutting services

- RoleService
- deep-link/password-recovery handling
- local storage/recovery-state handling
- SalePdfService
- direct printer integration when enabled

Only add another controller/repository/service to this master document after it exists or its schema/API has actually been finalized.

14. Supabase Responsibilities

Postgres / RPC

Supabase Postgres is the source of truth for shop, product, stock and transaction data.

Business-critical writes such as sale and purchase creation must stay atomic through the implemented database RPC/functions. Database functions should validate transaction input and update all dependent rows consistently.

Edge Functions (supabase/functions/.../index.ts)

Edge Function code is for operations that genuinely require trusted server-side credentials or privileged external/server behavior. It must not duplicate ordinary CRUD that is already safely handled by RLS + repositories/RPCs.

Any `index.ts` function documented here should correspond to an actual function in the project. Do not list speculative Edge Functions as part of the current architecture.

RLS

RLS is the real database authorization layer. Role checks in Flutter improve UX but are not a security boundary.

Policies must use the current membership/profile/shop schema and real column names. A policy copied from an older architecture is invalid if it references columns/tables that no longer exist.

At minimum, policies must enforce the current user's active membership and role for protected operations, while RPC permissions must align with the same role model.

15. Session & GetX State Safety

Authentication changes can leave old GetX controllers alive with data belonging to the previous user. Therefore logout, account switching and recovery entry must clear/reset user-scoped controller state.

Examples of user-scoped state that must not leak across accounts include product lists, home/dashboard data, sales, purchases, reports, expenses and shop-specific state.

The auth session is not sufficient by itself to decide whether cached controller data is valid.

16. UI / Theme Baseline

StockPulse uses a consistent light/dark theme through `AppThemeHelper` and shared constants/widgets.

Current design tokens used throughout the project include:

primary	#3D7BFF
secondary	#00C6FF
background	#F8FAFF
surface	#FFFFFF
textPrimary	#0F172A

textSecondary	#64748B
success	#10B981
error	#EF4444
border	#E2E8F0

Typography currently uses Google Fonts including Sora, Plus Jakarta Sans and Inter.

Reusable layout/style decisions already used in the app should be preferred over introducing a parallel design system.

17. Reliability Rules

- Never show transaction success before the Supabase/RPC call succeeds.
- Prevent duplicate submission while a write is in progress.
- Do not directly mutate stock from UI code.
- Keep sale/purchase stock effects atomic.
- Validate quantity, prices and required fields before submission and again server-side where business integrity depends on them.
- Use the actual creator ID for audit/display attribution.
- Clear user-scoped state on logout/account context changes.
- Treat expired password-recovery links as errors, not normal sessions.
- RLS policies must use actual current table/column types.
- Use numeric-compatible values for quantities/prices where fractional stock is supported.

18. Current Functional Scope

Implemented / actively integrated

- Supabase authentication
- Login/logout
- Forgot/reset password with HTTPS App Links
- Recovery-state/cold-start protection
- Shop setup/profile data
- Owner/staff membership and centralized RoleService
- Role-based dashboard/navigation
- Categories/products
- Product images
- Stock quantities and low-stock logic
- Sales + sale items + server RPC
- Purchases + purchase items + server transaction flow
- Stock movement architecture
- Expenses
- Dashboard/home data
- Sales reports
- Purchase reports

- Profit/report UI work
- Stock reports
- Invoice PDF/print flow
- GetX state management
- Supabase RLS/security work

Do not treat as finalized merely because it appeared in the old architecture

The old document contained fully specified Customers/Khata, Suppliers, activity_logs, a shop_profile singleton, a separate service layer for every repository, dev/prod project naming, an 18-day implementation schedule and other proposal-stage material. Those items are intentionally not authoritative in this current document unless/when their real StockPulse implementation is supplied and finalized.

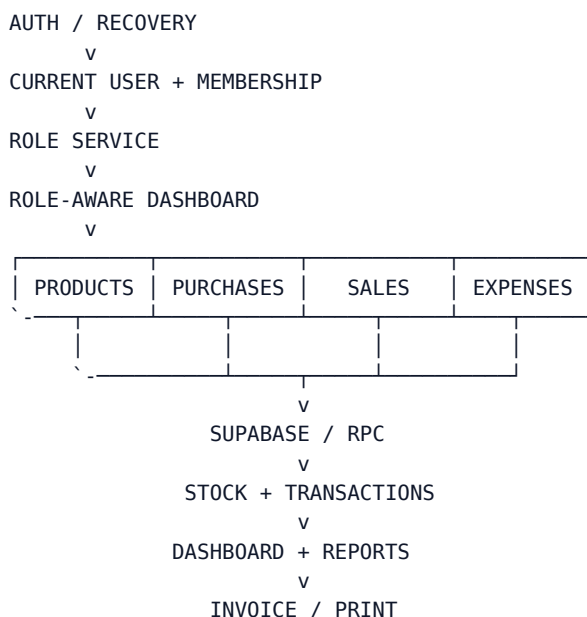
19. Current Source-of-Truth Principle

For StockPulse development, precedence should be:

1. Current Supabase schema / functions / RLS
2. Current Flutter models + repositories + controllers
3. Current implemented UI flow
4. This architecture document
5. Old planning/proposal documents

If this document conflicts with working code or the current database, inspect the implementation and update this document instead of changing working code merely to match an obsolete design note.

20. Final Current Flow



This is the maintained baseline for the current StockPulse app. It intentionally removes architecture that was speculative, superseded, duplicated or not yet implemented.